Practical Assignment 2

Team: Minnie & Fran

25th Nov 2025

1.1 AE vs VAE vs DAE (theory + diagram)

Part 1: Theoretical comparison: AE vs VAE vs DAE
(TL = transfer learning)

Aspect	AE	VAE	DAE
Latent type	Deterministic vector `z`	Probabilistic: `z ~ N(μ, σ²)`	Deterministic (same as AE)
Encoder output	`z`	`mu`, `logvar` (and then `z` via reparameterization)	`z`
Loss function	MSE per pixel	MSE per pixel + β · KL(N(μ,σ²)	
Input during training	Clean image `x`	Clean image `x`	Noisy image `x_noisy`
Reconstruction target	Reconstruct `x`	Reconstruct `x`	Reconstruct the clean `x` from noisy input
Conv. architecture (Enc/Dec)	Shared: same convolutional encoder–decoder	Shared: same conv encoder/decoder, extra linear for logvar	Shared: same as AE
Main goal	Compress and reconstruct images	Learn a structured, generative latent space	Learn noise-robust representations

1.1 AE vs VAE vs DAE (theory + diagram)

Part 2: Architectures of AE, VAE and DAE
All three models share the same **convolutional encoder–decoder backbone**.
They differ only in **what happens in the bottleneck (latent space)** and in **how the loss is defined**.

Encoder

Step	Layer	Main parameters	Output shape
1	Conv2d	in=1, out=16, kernel=3, stride=1, pad=1	16×28×28
2	ReLU	–	16×28×28
3	MaxPool2d	(2, 2)	16x14x14
4	Conv2d	in=16, out=32, kernel=3, stride=1, pad=1	32×14×14
4	ReLU	–	32×14×14
	MaxPool2d	(2, 2)	32×7×7
5	Flatten	32·7·7 → 1568	1568
6	Linear (μ)	1568 → latent_dim	latent_dim=32

The output of the last linear layer is the **bottleneck** (latent representation)

Decoder

Step	Layer	Main parameters	Output shape
1	Linear	latent_dim → 32·7·7	1568
2	Unflatten	1568 → (32, 7, 7)	32×7×7
3	ConvTranspose2d	in=32, out=16, kernel=3, stride=2, pad=1, out_pad=1	16×14×14
4	ReLU	–	16×14×14
5	ConvTranspose2d	in=16, out=1, kernel=3, stride=2, pad=1, out_pad=1	1×28×28
6	Sigmoid	– (forces output to [0,1])	1×28×28

This is the architecture of the **plain AE**.
VAE and DAE reuse the same blocks, changing only the bottleneck behaviour and the input/target.

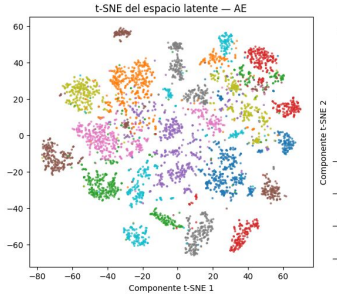
1.2 Hyperparameters

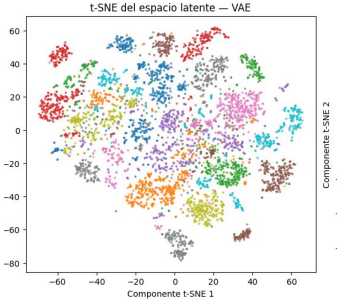
Hyperparameters

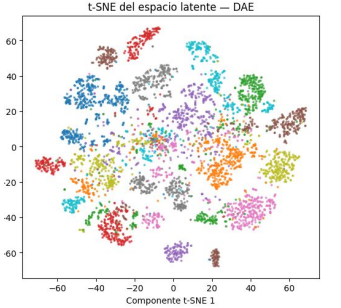
Category	Hyperparameter	Value	Comment
Model (latent space)	latent_dim	32	Dimension of the bottleneck / latent vector.
Data / DataLoader	batch_size	100	Batch size for AE, VAE, DAE and classifier.
AE training	epochs_ae	20	Number of epochs for the Autoencoder.
VAE training	epochs_vae	20	Number of epochs for the Variational AE.
DAE training	epochs_dae	20	Number of epochs for the Denoising AE.
Classifier training	epochs_clf	20	Epochs for the MLP classifier on top of encoder.
Optimization	lr	1e-3	Learning rate for all models (Adam).

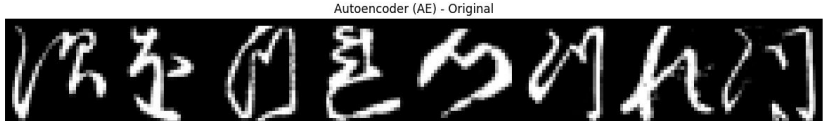
Category	Hyperparameter	Value	Comment
VAE regularisation	beta_kl	1.0	Weight of the KL term in the VAE loss.
DAE regularisation	noise_factor	0.5	Std factor of the Gaussian noise added to inputs.
Classifier architecture	hidden_units	128	Size of the hidden layer in the classifier MLP.
Classifier regularisation	dropout_p	0.3	Dropout probability in the classifier's hidden layer.
t-SNE	perplexity	30 (default)	Main hyperparameter for t-SNE latent visualisation.
t-SNE / sampling	max_tsne_samples	≈ 5000 samples	Limit on the number of latent points used for t-SNE.

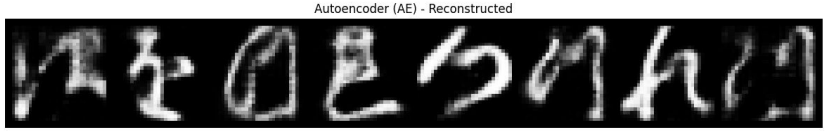
1.3 Plots

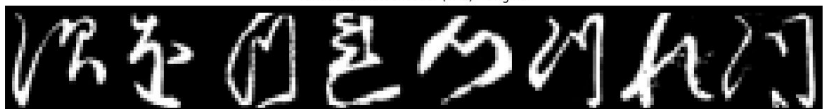

t-SNE del espacio latente — AE
Componente t-SNE 1
Componente t-SNE 2


t-SNE del espacio latente — VAE
Componente t-SNE 1
Componente t-SNE 2

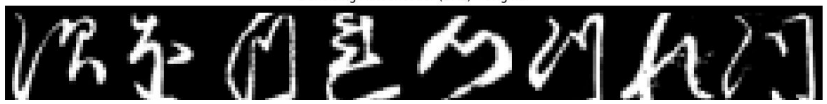

t-SNE del espacio latente — DAE
Componente t-SNE 1


Autoencoder (AE) - Original


Autoencoder (AE) - Reconstructed

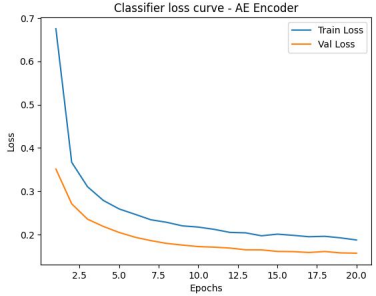

Variational Autoencoder (VAE) - Original

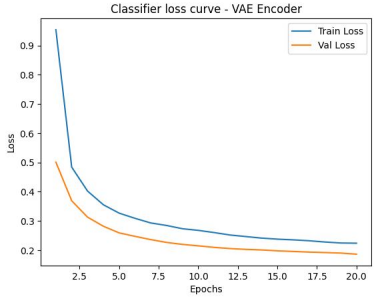

Variational Autoencoder (VAE) - Reconstructed

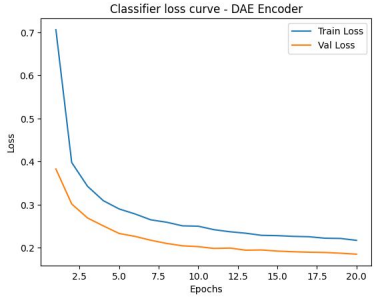

Denoising Autoencoder (DAE) - Original


Denoising Autoencoder (DAE) - Noisy


Denoising Autoencoder (DAE) - Reconstructed


Classifier loss curve - AE Encoder
Train Loss
Val Loss
Loss
Epochs


Classifier loss curve - VAE Encoder
Train Loss
Val Loss
Loss
Epochs


Classifier loss curve - DAE Encoder
Train Loss
Val Loss
Loss
Epochs

1.4 Classification architectures used and results obtained

Part 1: Classification architectures used (transfer learning)

For all three encoders, we use **exactly the same classifier**, changing only which encoder produces the latent vector

Encoder (frozen):

AE / DAE: z = encoder.encode(x)

VAE: mu, logvar = encoder.encode(x) and we use z = mu

Latent dimension: latent_dim = 32

Classifier MLP (trained from scratch):

z (dim = 32)

→ Linear(32 → 128)
→ ReLU
→ Dropout(p = 0.3)
→ Linear(128 → 10)
→ (Softmax implicitly inside CrossEntropyLoss)

Training setup for the classifier:

Encoder weights **frozen** (requires_grad = False)

Optimizer: Adam, lr = 1e-3

Loss: CrossEntropyLoss

Epochs: epochs_clf = 20

Same train/val/test split as in the autoencoder training.

Part 2: Classification results using each encoder's weights

The following table summarizes the **test results** when training the same classifier on top of each encoder:

Frozen encoder used	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
AE encoder	≈ 0.8913	≈ 0.8924	≈ 0.8813	≈ 0.8913
VAE encoder	≈ 0.8789	≈ 0.8809	≈ 0.8789	≈ 0.8789
DAE encoder	≈ 0.8726	≈ 0.8745	≈ 0.8726	≈ 0.8727

The **AE encoder** yields the **best performance** with this classifier:
Its latent space is highly tuned to reconstruct characters and ends up being quite discriminative for a small non-linear classifier.

The **VAE encoder** performs slightly worse:
The KL term regularizes the latent space to be smoother and more "Gaussian", which is good for generative behavior but slightly hurts pure classification performance.

The **DAE encoder** is close to the VAE:
The DAE is optimized for robustness to noise; its latent space is very stable, but in this experiment you are classifying clean test images, so that robustness does not fully translate into higher accuracy.

1.5 Comparison and detailed analysis of the results obtained

AE, VAE and DAE architectures

Reconstruction

AE (Autoencoder)

Achieves the best reconstruction on clean images.

It only optimises the reconstruction MSE, so the latent space is fully adapted to that objective.

VAE (Variational AE)

Reconstruction is slightly worse (images are a bit blurrier).

In addition to MSE, it includes a KL term that forces the latent space to approximate a N(0,I).
This regularisation limits how much information can be encoded and slightly worsens reconstruction.

DAE (Denoising AE)

Trained with noisy input and clean targets.

On clean data its MSE is similar or slightly worse than the AE, but when noise is added at test time, its reconstructions are clearly more robust.

Latent space structure (t-SNE)

AE

t-SNE shows well-separated clusters by class.

Even though it is not trained for classification, the latent space becomes quite discriminative.

VAE

t-SNE shows more mixing between colours (overlapping classes).

The KL term makes all classes share a smooth, continuous manifold, which is more useful for generation and interpolation than for perfect class separation.

DAE

Shows some structure and grouping, but not as clean as the AE.

Its main goal is noise invariance, not maximising separation between classes.

Transfer learning (classification with frozen encoder)

Interpretation:

The AE provides the most discriminative latent space for this specific classification task.

The VAE trades some classification performance for a more regular, generative latent space.

The DAE is specialised in robustness to noise, not in maximising accuracy on clean data, although it remains competitive.

Global conclusion

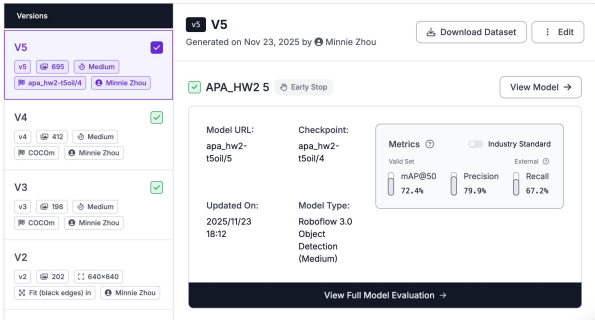
In summary, the AE is the best feature extractor for classification, the VAE is the most interesting from a generative perspective (thanks to its regularised latent space), and the DAE is the most robust to noise. The choice between them depends on whether we prioritise reconstruction and classification, generative capabilities, or robustness.

2.1 Annotation Process

Annotation Tool: Roboflow platform

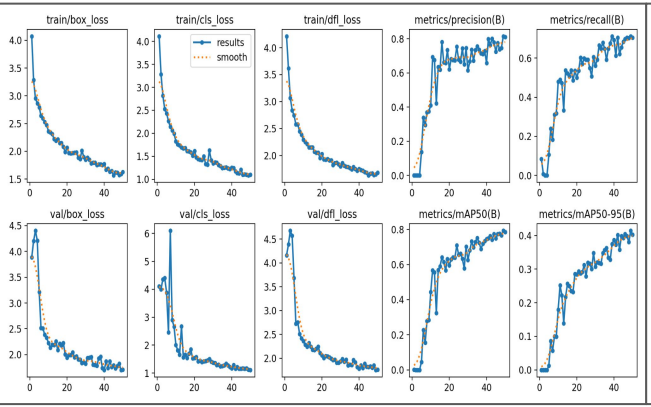
Step by step:

1. Baseline Manual Labeling: A total of **82 images** were manually labeled from scratch;
2. Dataset Version Creation: a) Preprocessing: All images were resized using `Fit (black edges) in 640x640`
   b) Augmentation: Augmentation was applied (e.g., Horizontal Flip, Exposure, Rotation) to the training set.
1. Training Set Split for Roboflow: To satisfy the platform's mandatory requirement for a **Train/Valid/Test** split, the 82 manually labeled images were partitioned: **Training Set: 66 images**, **Validation Set: 14 images**, **Testing Set: 2 images**.
2. Baseline Model Training: A "mini-model" was trained using the dataset version:
   - Architecture: Roboflow 3.0 (YOLOv8-compatible)
   - Size: Medium
   - Weights: Trained from **Public Checkpoint (MS COCO)** to leverage transfer learning.
   (V4-VERSION , mAP@50(79.4%)
1. Auto-Labeling: The trained Baseline Model was run on the remaining unlabeled images using Roboflow's **Label Assist** feature.
2. Human Refinement:
   - Removal of False Positives (incorrect boxes).
   - Adjustment of inaccurate box positions and dimensions.
   - Addition of False Negatives (missed human instances).
1. After the first cycle of auto-labeling and refinement, the dataset was expanded from 82 images to **224 images**.
1. Retraining Iteration (V5)
   New Dataset Size: 224 Images
   New Dataset Split: split was adopted: Train: 157 images, Valid: 45 images, Test: 22 images.(70/20/10)
   Training Strategy: The Train from Previous Checkpoint method was used.
   Model Type: Roboflow 3.0 Medium
   V5-VERSION , mAP@50(72.4%) *This decline does not signify poorer model performance; rather, it reflects a more stringent evaluation. By increasing the size of our validation set, we introduced more challenging and diverse images, making the 72.4% a more robust and accurate assessment of the model's true generalization ability.*
1. Human Refinement until all **600 images** were fully annotated. (The dataset mainly contains 1,718 human instances.)

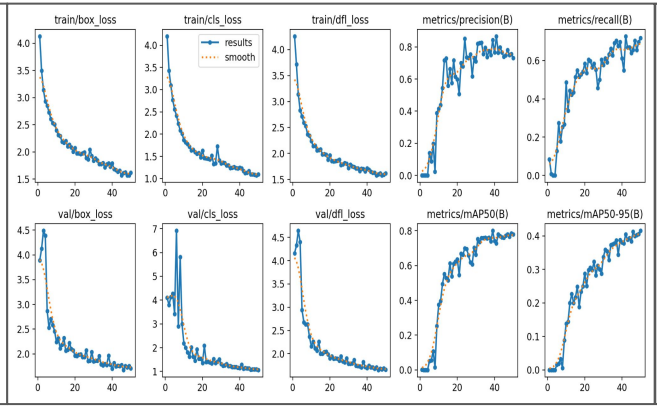


2.2 Analysis of the results

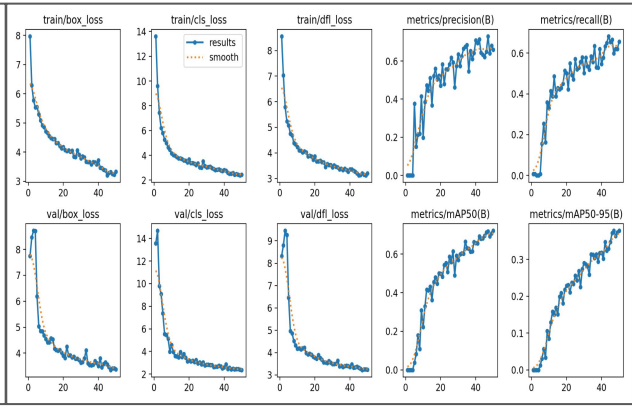
YOLOv8s(scratch)



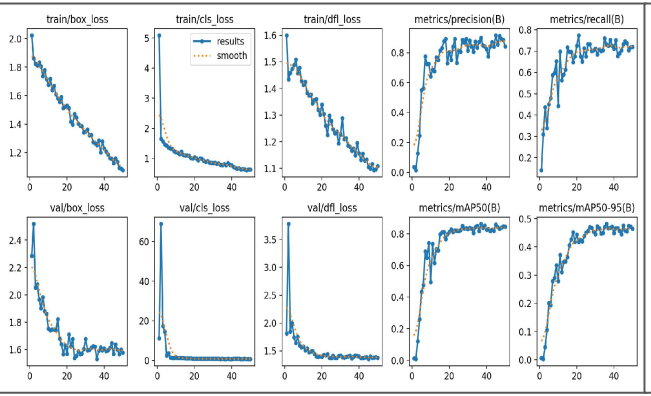
YOLOv9s(scratch)



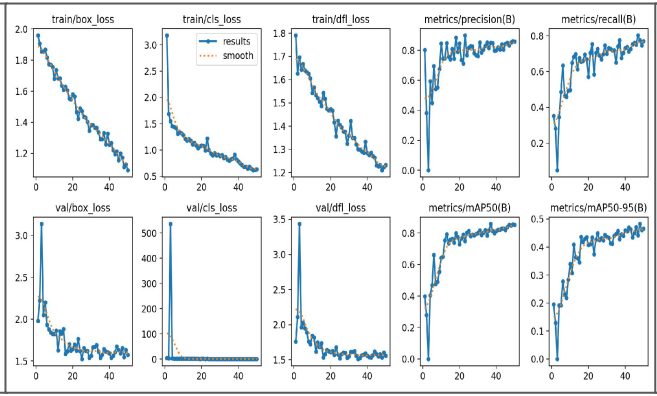
YOLOv10s(scratch)



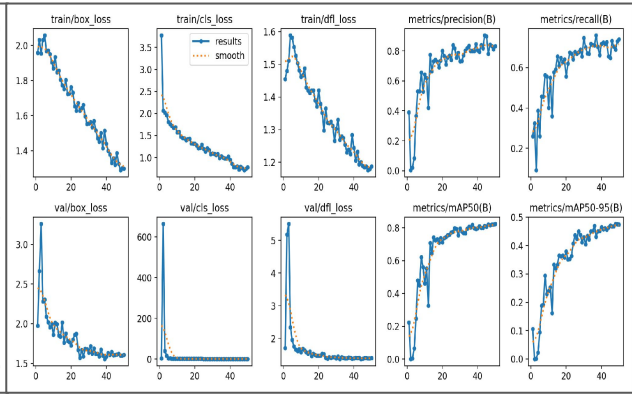
YOLOv8s(pretrained)



YOLOv9s(pretrained)

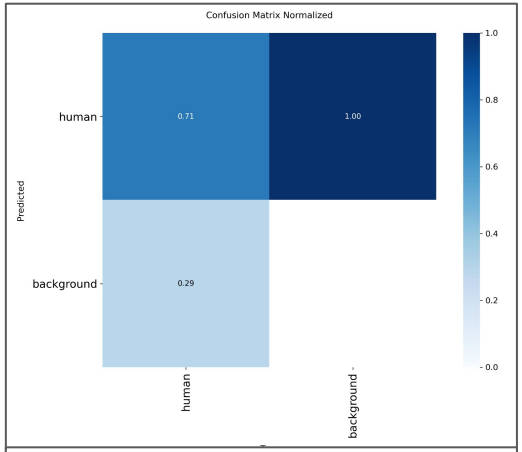


YOLOv10s(pretrained)

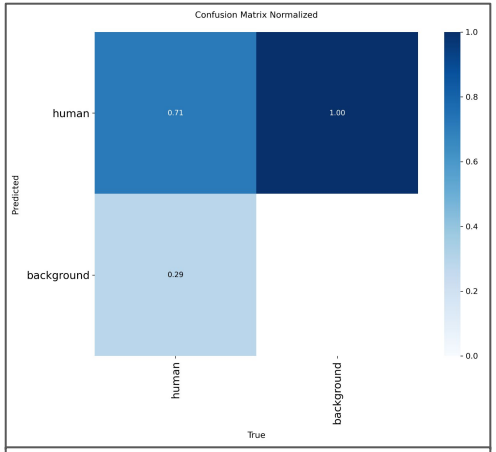


2.2 Analysis of the results

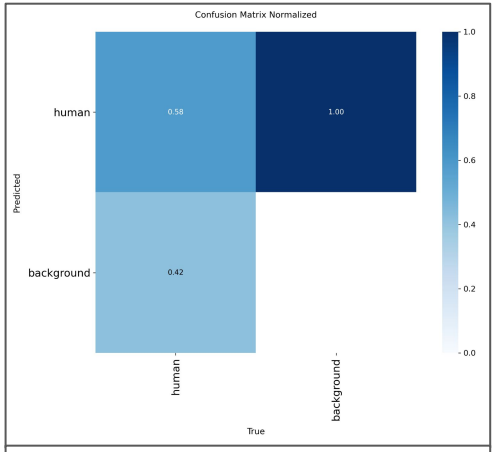
YOLOv8s (scratch)



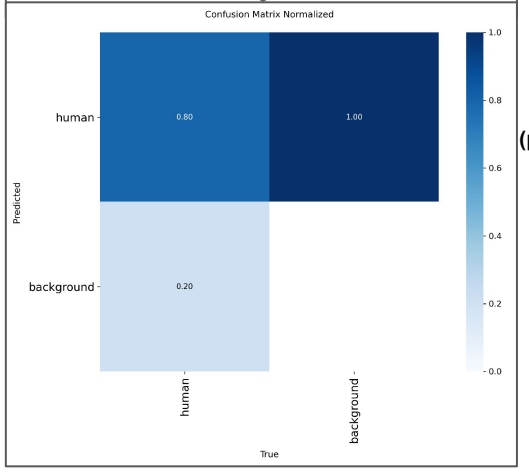
YOLOv9s (scratch)



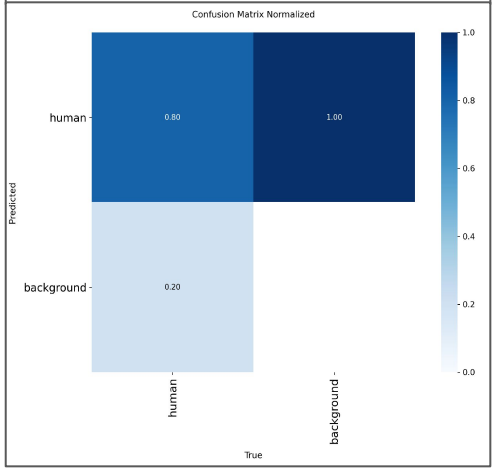
YOLOv10s (scratch)



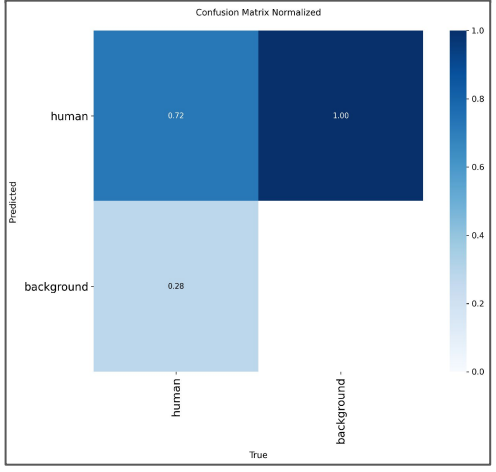
YOLOv8s (pretrained)



YOLOv9s (pretrained)



YOLOv10s (pretrained)



2.2 Analysis of the results

Part 2 evaluated 6 YOLO model(YOLOv8s, YOLOv9s, and YOLOv10s) each trained both from scratch and using pretrained weights. Several key performance metrics were observed.

What we see in the loss/metric plots

- **Training loss (box / cls / dfl)**: smooth decrease across epochs, so training is stable and not diverging.
- **Validation loss**: generally falls but sometimes spikes early on, probably because a few hard examples in the val set cause temporary bumps. Overall there's no clear sign of significant overfitting.
- **Precision / Recall / mAP**: they increase with epochs. Pretrained models show faster and higher gains, which means pretraining helps a lot on your dataset.

What the confusion matrices tell us

- For scratch models the true-positive rate for the "human" class is around ~0.58–0.71 depending on model. After using pretrained weights it increases to roughly ~0.72–0.80.
- False positives on "background" are reduced in pretrained versions. So pretraining makes the model better at both finding humans and not mistaking background for humans.

Summary table (test results)

Model	training time (h)	mAP50 (Test)	mAP50-95 (Test)
YOLOv8s_scratch	0.177	0.7574	0.3748
YOLOv8s_pretrained	0.179	0.8264	0.4497
YOLOv9s_scratch	0.223	0.7461	0.3693
YOLOv9s_pretrained	0.220	0.8237	0.4278
YOLOv10s_scratch	0.210	0.6465	0.3283
YOLOv10s_pretrained	0.201	0.7940	0.4094

Result

1. Pretrained models outperform scratch models for every architecture tested.
2. The **YOLOv8s Pretrained** model achieved the highest mAP50-95 of 0.4497.
3. In terms of training time, the YOLOv8s architecture is the fastest, whether trained from scratch (0.177 h) or with pretraining (0.179 h), indicating that its architecture is the most efficient during the learning iterations.

2.3 comparison

The analysis demonstrates that Transfer Learning is the single most critical factor in achieving object detection performance on this dataset. The performance increase brought by pretraining is significant across all metrics and architectures tested.

Model	mAP50 (scratch → pretrained)	mAP50 (abs)	mAP50 (%)	mAP50-95 (scratch → pretrained)	mAP50-95 (abs)	mAP50-95 (%)
YOLOv8s	0.7574 → 0.8264	+0.0690	**+9.11%**	0.3748 → 0.4497	+0.0749	**+19.99%**
YOLOv9s	0.7461 → 0.8237	+0.0776	**+10.40%**	0.3693 → 0.4278	+0.0585	**+15.84%**
YOLOv10s	0.6465 → 0.7940	+0.1475	**+22.81%**	0.3283 → 0.4094	+0.0811	**+24.70%**

1.Transfer learning demonstrated consistent and substantial improvements across all architectures
All models achieved substantial gains in mAP50-95, the use of pretrained weights led to significant performance boosts. This confirms that pretrained features provide a strong initialization, enabling models to converge more effectively on the target dataset.
2.The magnitude of improvement varied significantly across architectures:
Robust Baseline (YOLOv8s, YOLOv9s): Both YOLOv8s and YOLOv9s, while achieving strong final performance, showed mAP50-95 gains of +19.99% and +15.84%. Their Scratch versions were already starting from a more robust baseline (mAP50-95 around 0.37),indicating strong inherent feature-learning capabilities even without pretrained initialization.
Highest Dependency(YOLOv10s): The YOLOv10s architecture showed the most dramatic increase in performance. Its mAP50-95 increased by 0.0811 absolute points (a relative gain of +24.70%). This indicates that YOLOv10s sacrifices its ability to build foundational features from scratch on a specialized dataset, making it highly dependent on the pre-trained weights for initialization.
3.These findings highlight that while all models benefit from transfer learning, architectures like YOLOv10s may be unsuitable for small-data scenarios when trained from scratch. In contrast, YOLOv8s and YOLOv9s offer more stable baseline performance and robustness in low-data environments.